



THE STATE UNIVERSITY OF ZANZIBAR (SUZA)
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Name: Citizen Alert System

Version:1.0

Date: February 2026

Author(s):

Zahor Khalfan Omar (24BCS018)

Khadija Ali Said (24BCS026)

Abdulkadir Bakar Mbwana (24BCS028)

Status: Draft

1. INTRODUCTION

1.1 Purpose

The purpose of this document is to clearly describe the requirements for the Citizen Alert System. This is a **command-line based system** developed in C programming language. It is intended for the project team, supervisors, and stakeholders. This document will guide the development of the **terminal-based application** to ensure the system meets the needs of citizens and government officials.

1.2 Project Scope

The Citizen Alert System is a **command-line interface (CLI) digital platform** that allows government institutions to send verified time-sensitive alerts to citizens. It includes:

- A **terminal-based interface** for citizens to register, view alerts, and search for citizens
- A **terminal-based administration portal** for government officials to create, manage, and remove announcements
- **Role-based access control** with Super Admin and Admin roles
- **Priority-based notification queuing** with sorting by urgency (Low, Normal, High, EMERGENCY)
- **Targeted alerts** based on region, citizen status (Student/Worker/General), or education level

1.3 Definitions and Acronyms

- **SRS:** Software Requirements Specification
- **UI:** User Interface
- **API:** Application Programming Interface
- **GPS:** Global Positioning System
- **Admin:** System administrator or government official
- **User:** Citizen receiving alerts
- **CLI:** Command Line Interface
- **OTP:** One-Time Password (in this C implementation, simplified registration without OTP)
- **Queue:** Data structure used for notification storage and priority sorting
- **Super Admin:** System administrator with full account management privileges
- **Admin:** Government official who can only manage their own office's notifications

2. OVERALL DESCRIPTION

2.1 Product Perspective

The Citizen Alert System is a **standalone command-line application** written in C. It connects citizens with government institutions through a terminal interface. The system uses **linked lists** for data storage and a **queue data structure** for notification management. All data is stored in memory during runtime (no persistent database in this version).

2.2 Product Features

The system includes the following key features:

- **Priority-based notifications:** Alerts sorted by urgency (Emergency > High > Normal > Low)
- **Targeted alerts:** Citizens receive alerts based on region, status, or minimum education level
- **Role-based authentication:** Super Admin and Admin roles with different permissions
- **Office account management:** Super Admin can create, list, toggle, delete, and reset passwords for office accounts
- **Citizen registration:** Citizens register with name, region, status, and education level
- **Notification management:** Create, list, search, and remove notifications
- **Emergency override:** Emergency priority alerts bypass all targeting filters
- **Multi-level targeting:** All citizens, specific region, specific status, or minimum education level
-

2.3 User Classes and Characteristics

- **Super Administrator:**
 - Full system access including account management
 - Can manage all notifications across all offices
 - High technical skill level (terminal/CLI comfortable)
- **Admin (Government Official):**
 - Manages notifications only for their own office
 - Cannot manage other office accounts
 - Medium technical skill level
- **Citizen (End User):**
 - Registers and views alerts
 - No authentication required after registration (uses Citizen ID)
 - Low technical skill level (simple menu navigation)

2.4 Operating Environment

- **Platform:** Any system with a C compiler (Linux, macOS, Windows with GCC/MinGW)
- **Terminal:** Any standard terminal or command prompt
- **Hardware:** Minimal requirements (runs on any system with 64MB RAM)
- **Network:** No network required (offline/local system)
- **Dependencies:** Standard C libraries only (stdio.h, stdlib.h, string.h, time.h)

2.5 Design and Implementation Constraints

- Must compile with standard C compilers (GCC, Clang, MSVC)
- No external database - data stored in memory using linked lists
- No network connectivity - fully offline system
- Input validation for all user inputs
- Priority queue implemented with bubble sort on linked list
- Maximum string lengths: 64 chars for names, 256 chars for messages

2.6 Assumptions and Dependencies

- Users can operate a command-line/terminal interface
- Government officials have access to a computer with terminal
- First user automatically becomes Super Admin
- No persistent storage - data lost when program terminates
- Citizen IDs are auto-generated starting from 100
- Office account IDs are auto-generated starting from 1
- Notification IDs are auto-generated starting from 1

3. FUNCTIONAL REQUIREMENTS

3.1 Feature 1: User Registration and Authentication

FR-001: Office Account Creation (Super Admin only)

- **Description:** Super Admin can create office accounts with username, password, office name, and role (Admin or Super Admin)
- **Priority:** High
- **Input:** Username, password, office name, role selection (0=Admin, 1=Super Admin)
- **Processing:** System validates unique username, stores in linked list with auto-generated ID
- **Output:** Account created with unique ID
- **Acceptance Criteria:** New office account can log in successfully

FR-002: Office Login

- **Description:** Government officials can log in using username and password
- **Priority:** High
- **Input:** Username, password
- **Processing:** System searches linked list for matching credentials, checks if account is active
- **Output:** Access granted to role-specific menu (Super Admin or Admin)
- **Acceptance Criteria:** Only active accounts with correct password can access the system

FR-003: Citizen Registration

- **Description:** Citizens can register without authentication (no OTP in current version)
- **Priority:** High
- **Input:** Name, region, status (0=Student,1=Worker,2=General), education level (0-5)
- **Processing:** System auto-generates Citizen ID starting from 100
- **Output:** Citizen ID displayed for future alert viewing
- **Acceptance Criteria:** Citizen can view alerts using their assigned ID

FR-004: Account Management (Super Admin only)

- **Description:** Super Admin can list, toggle active status, delete, and reset passwords for office accounts
- **Priority:** High
- **Input:** Account ID for target operation
- **Processing:** System modifies the office account linked list
- **Output:** Confirmation of operation
- **Acceptance Criteria:** Super Admin cannot delete their own account

3.2 Feature 2: Announcement Management

FR-005: Create Notification

- **Description:** Admins create notifications with title, message, priority (0-3), and target type (All/Region/Status/Education)
- **Priority:** High
- **Input:** Title, message, priority (0=Low,1=Normal,2=High,3=Emergency), target type, target value
- **Processing:** System stores notification in queue with timestamp and office name
- **Output:** Notification created with unique ID
- **Acceptance Criteria:** Emergency priority alerts bypass all targeting filters

FR-006: List Notifications

- **Description:** Admins view all active notifications for their office (Super Admin sees all)
- **Priority:** High
- **Input:** None (uses logged-in admin's office/role)
- **Processing:** System traverses queue and displays notifications sorted by priority
- **Output:** Table of notifications with ID, Title, Office, Priority, Target
- **Acceptance Criteria:** Only notifications from admin's own office shown for regular admins

FR-007: Search Notification

- **Description:** Any user can search for a specific notification by ID
- **Priority:** Medium
- **Input:** Notification ID
- **Processing:** System traverses queue to find matching ID
- **Output:** Full notification details including title, message, office, priority, target, timestamp
- **Acceptance Criteria:** Shows message content even for inactive notifications

FR-008: Remove Notification

- **Description:** Admins can deactivate notifications (soft delete)
- **Priority:** Medium
- **Input:** Notification ID
- **Processing:** System sets notification active flag to 0
- **Output:** Confirmation of removal
- **Acceptance Criteria:** Regular admins can only remove notifications from their own office

3.3 Feature 3: Notification System

FR-009: View My Alerts (Citizen)

- **Description:** Citizens view alerts matching their profile (region, status, education level)
- **Priority:** High
- **Input:** Citizen ID
- **Processing:** System validates citizen exists, then filters active notifications based on:
 1. Emergency priority: always shown
 2. Region target: matches citizen's region
 3. Status target: matches citizen's status
 4. Education target: citizen's level $\geq$ minimum required
 5. All target: always shown
- **Output:** Sorted list of relevant alerts with title, office, timestamp, and message
- **Acceptance Criteria:** Only matching alerts are displayed, sorted by priority

FR-010: Priority Sorting

- **Description:** System automatically sorts notifications by priority (Emergency > High > Normal > Low)
- **Priority:** High
- **Input:** None (automatic before display)
- **Processing:** Bubble sort algorithm on notification queue
- **Output:** Notifications displayed in priority order
- **Acceptance Criteria:** Emergency alerts always appear first

FR-010: Priority Sorting

- **Description:** Users can search for citizens by ID or name
- **Priority:** Low
- **Input:** Search type (1=ID, 2=Name), search value
- **Processing:** System traverses citizen linked list
- **Output:** Citizen details (ID, name, region, status, education)
- **Acceptance Criteria:** Partial name matching using substring search

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

- **NFR-P01:** The system shall respond to user requests within 1 second (C is fast!)
- **NFR-P02:** The system shall support at least 10,000 citizens in linked list without significant slowdown
- **NFR-P03:** Notification queue sorting (bubble sort) shall complete within 3 seconds for up to 1000 notifications

4.2 Security Requirements

- **NFR-S01:** Passwords shall be stored as plain text (note: not encrypted - improvement needed)
- **NFR-S02:** No session timeout implemented (user must manually logout)
- **NFR-S03:** Admin access requires role-based authorization (Super Admin vs Admin)
- **NFR-S04:** No data encryption in transit (offline system)
- **NFR-S05:** Input validation required for all numeric inputs to prevent crashes

4.3 Usability Requirements

- **NFR-U01:** The system shall provide clear menu-driven interface with box-drawing characters
- **NFR-U02:** New users shall be able to complete basic tasks within 5 minutes
- **NFR-U03:** The system shall display status/education as readable strings (e.g., "Student" not "0")
- **NFR-U04:** Input validation with clear error messages for invalid entries

4.4 Reliability Requirements

- **NFR-R01:** The system shall handle invalid inputs without crashing (robust error handling)
- **NFR-R02:** No data persistence - data lost when program terminates (limitation to document)
- **NFR-R03:** Memory leaks shall be prevented by proper free() calls on program exit
- **NFR-R04:** The system shall validate all pointer operations before dereferencing

5. USER STORIES

5.1 User Story 1 - Office Account Creation

- **As a** Super Administrator
- **I want to** create office accounts for government officials
- **So that** different government offices can send announcements

Acceptance Criteria:

1. Given a Super Admin, when they create an account, then the account is added to the linked list
2. Given a duplicate username, when creation is attempted, then the system rejects it
3. Given a new account, when the user logs in, then they see the appropriate role menu

5.2 User Story 2 - Citizen Registration

- **As a** citizen
- **I want to** register using my name, region, status, and education level
- **So that** I can receive targeted government alerts

Acceptance Criteria:

1. Given a new citizen, when they complete registration, then they receive a unique Citizen ID
2. Given a registered citizen, when they view alerts, then they only see matching notifications

5.3 User Story 3 - Create Targeted Notification

- **As an** administrator
- **I want to** create notifications with priority levels and targeting rules
- **So that** citizens receive relevant and urgent information

Acceptance Criteria:

1. Given an admin, when they create a notification, then it is added to the queue
2. Given Emergency priority, when notification is created, then it bypasses all targeting
3. Given Region targeting, when citizens view alerts, then only citizens in that region see it

5.4 User Story 4 - View Personalized Alerts

- **As a** citizen
- **I want to** view alerts using my Citizen ID
- **So that** I only see announcements relevant to my profile

Acceptance Criteria:

1. Given a citizen with ID 100, when they view alerts, then only matching notifications appear
2. Given multiple notifications, when displayed, then they are sorted by priority
3. Given an invalid ID, when viewing alerts, then the system shows "Citizen not found"

6. USE CASES

6.1 Use Case 1 - Citizen Registration

- **Use Case ID:** UC-001
- **Name:** Citizen Registration
- **Actor(s):** Citizen
- **Description:** Citizen registers with profile information to receive targeted alerts
- **Preconditions:** System is running

Main Flow:

1. Citizen selects "Register" from Citizen Portal menu
2. System prompts for name
3. Citizen enters name
4. System prompts for region
5. Citizen enters region
6. System prompts for status (0=Student,1=Worker,2=General)
7. Citizen enters status number
8. System prompts for education level (0-5)
9. Citizen enters education level
10. System auto-generates Citizen ID and stores in linked list
11. System displays Citizen ID to user

Alternative Flows:

- If invalid status or education input, system rejects and prompts again

Postconditions:

- Citizen can view alerts using their ID

6.2 Use Case 2 - Receive Targeted Alerts

- **Use Case ID:** UC-002
- **Name:** Receive Targeted Alerts
- **Actor(s):** Citizen
- **Description:** Citizen views alerts filtered by their profile and sorted by priority
- **Preconditions:** Citizen is registered and has Citizen ID

Main Flow:

1. Citizen selects "View My Alerts" from Citizen Portal menu
2. System prompts for Citizen ID
3. Citizen enters ID
4. System validates citizen exists
5. System sorts all active notifications by priority
6. System filters notifications based on citizen's:
 1. Region (if target type = Region)
 2. Status (if target type = Status)
 3. Education level (if target type = Education)
7. System displays matching notifications with full details
8. Emergency priority alerts are always shown regardless of targeting

Alternative Flows:

- If Citizen ID not found, system displays error message
- If no matching alerts, system displays "No alerts for you right now"

Postconditions:

- Citizen has viewed relevant alerts

6.3 Use Case 3 - Admin Create Announcement

- **Name:** Create and Publish Announcement
- **Actor(s):** Administrator (Super Admin or Admin)
- **Description:** Admin creates a notification with priority and targeting rules
- **Preconditions:** Admin is logged into the system

Main Flow:

1. Admin selects "Create Notification" from menu
2. System prompts for title
3. Admin enters title
4. System prompts for message
5. Admin enters message
6. System prompts for priority (0-3)
7. Admin enters priority
8. System prompts for target type (0=All,1=Region,2=Status,3=Education)
9. Admin selects target type
10. If Region: System prompts for region name, Admin enters region
11. If Status: System prompts for status number, Admin enters value
12. If Education: System prompts for minimum education level, Admin enters value
13. System auto-generates Notification ID and timestamp
14. System stores notification in queue with admin's office name
15. System displays "Notification created" with ID

Alternative Flows:

- If invalid priority, system defaults to Normal
- If invalid target type, system defaults to All

Postconditions:

- Notification is queued and will appear in citizens' alerts

7. REQUIREMENTS TRACEABILITY MATRIX

Purpose: The RTM ensures every requirement is tracked from user story -- test case -
- implementation

Req ID	Requirement Description	User Story	Status
FR-001	Office Account Creation	US-001	Implemented
FR-002	Office Login	US-002	Implemented
FR-003	Citizen Registration	US-003	Implemented
FR-004	Account Management (Super Admin)	US-004	Implemented
FR-005	Create Notification	US-005	Implemented
FR-006	List Notifications	US-006	Implemented
FR-007	Search Notification	US-007	Implemented
FR-008	Remove Notification	US-008	Implemented
FR-009	View My Alerts	US-009	Implemented
FR-010	Priority Sorting	US-010	Implemented
FR-011	Citizen Search	N/A	Implemented

8. APPROVAL

This SRS has been updated to reflect the actual C command-line implementation. The original SRS described a mobile app with web dashboard, but the delivered system is a terminal-based application due to scope and technical constraints.

- **Project Supervisor:** Mr. Massoud Hamad Mmanga
 - Signature / Date: _____
- **Team Lead:** Zahor Khalfan Omar
 - Signature / Date: 03/02/2026

CONCLUSION.

The Software Requirements Specification (SRS) for the Citizen Alert System provides a clear description of the **command-line based C implementation**. It defines the functional requirements (office account management, citizen registration, priority-based notifications, targeted alerts), non-functional requirements (performance, security limitations, usability), user stories, and use cases that accurately reflect the delivered system.

Key Implementation Features:

- Linked lists for citizen and office account storage
- Queue data structure for notification management
- Priority sorting (Emergency > High > Normal > Low)
- Four targeting methods (All, Region, Status, Education)
- Role-based access (Super Admin and Admin)

Known Limitations:

- No persistent database (data lost on program exit)
- No password encryption
- No network functionality (offline only)
- No session timeout
- No OTP verification for citizens

Prepared by: Khadija Ali Said - System Analyst

Approved by:

- Team Manager: Zahor Khalfan Omar
- Developer: Abdulkadir Bakar Mbwana